

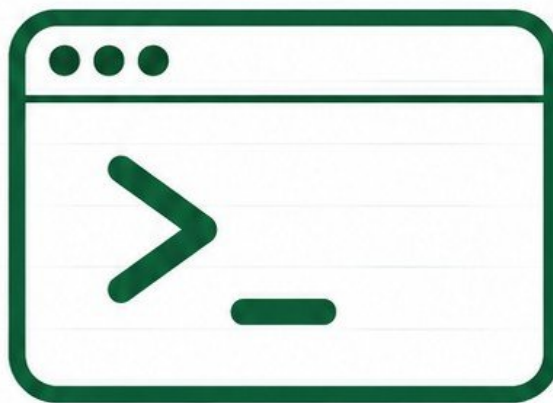


VERIQTA

VERIQTA

SHELL SCRIPTING FOR DEVOPS HANDBOOK

FOR COMPLETE BEGINNERS



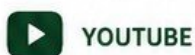
- ✓ LEARN
- ✓ PRACTICE
- ✓ AUTOMATE
- ✓ DEPLOY

✓ STEP-BY-STEP
EXPLANATIONS

✓ PRACTICAL
EXAMPLES

✓ DEVOPS
USE CASES

✓ BEGINNER
FRIENDLY



YOUTUBE



GITHUB



LINKEDIN



X



INSTAGRAM



TELEGRAM

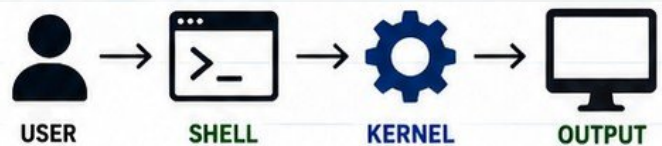
1. WHAT IS SHELL SCRIPTING?

SHELL VS TERMINAL VS BASH

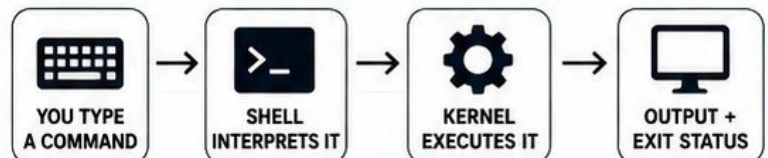
	TERMINAL	The application you open. It gives you a text interface to interact with the computer.
	SHELL	The program that interprets your commands and communicates with the operating system.
	BASH	The most common shell on Linux. "Bourne Again SHell". It is a command language and scripting interpreter.

WHAT A SHELL ACTUALLY DOES

- 1 Reads your command.
- 2 Parses (understands) the command.
- 3 Expands variables, wildcards, and paths.
- 4 Executes the command or program.
- 5 Returns the output and exit status.
- 6 Waits for the next command.



COMMAND EXECUTION FLOW



WHY DEVOPS ENGINEERS AUTOMATE WITH SCRIPTS

- ✓ Save time and reduce manual work.
- ✓ Eliminate human errors.
- ✓ Ensure repeatability and consistency.
- ✓ Scale operations easily.
- ✓ Automate deployments, monitoring, backups, and more.
- ✓ Integrate with CI/CD pipelines.
- ✓ Improve reliability and speed.

INTERACTIVE COMMANDS VS SCRIPTS

INTERACTIVE COMMANDS	SCRIPTS
• You type and run one command at a time. • Used for ad-hoc tasks. • Hard to repeat. • Not version controlled. • Easy to forget commands.	• You write commands in a file and run them. • Used for automation. • Repeatable and consistent. • Version controlled (Git). • Easy to share and reuse.

YOUR FIRST BASH SCRIPT

- | | | |
|---|---|--|
| 1 | <code>#!/bin/bash</code> | → Shebang: tells the system to use Bash. |
| 2 | <code># This is my first script</code> | → Comment: ignored by the shell. |
| 3 | <code>echo "Hello, DevOps Engineer!"</code> | → Prints a message to the screen. |
| 4 | <code>DATE=\$(date)</code> | → Stores the current date in a variable. |
| 5 | <code>echo "Today is: \$DATE"</code> | → Prints the date. |



IMPORTANT NOTE

A script has the same power as the user who runs it. Always review scripts before executing, especially scripts from the internet.

RUNNING A SCRIPT SAFELY

- 1  **CREATE THE FILE**
`nano myscript.sh`
- 2  **SAVE AND EXIT**
`CTRL + O, ENTER, CTRL + X`
- 3  **MAKE IT EXECUTABLE**
`chmod +x myscript.sh`
- 4  **RUN THE SCRIPT**
`./myscript.sh`
- 5  **BEST PRACTICES**
 - Review your script before running.
 - Use full paths for important commands.
 - Test in a safe environment first.

2. SHELL COMMANDS

YOU MUST KNOW FIRST

1. NAVIGATE YOUR FILE SYSTEM

pwd	Print working directory pwd /home/user
ls	List files and folders ls file1.txt docs images
cd [dir]	Change directory cd docs (moves into docs)
cd ..	Go up one directory cd .. (moves up one level)
cd ~	Go to your home directory cd ~ /home/user

2. CREATE, COPY, MOVE, DELETE

mkdir [dir]	Create a new directory mkdir new_folder (creates new_folder)
touch [file]	Create an empty file touch notes.txt (creates notes.txt)
cp [src] [dest]	Copy files or folders cp file1.txt backup.txt (copies file1.txt)
mv [src] [dest]	Move or rename files/folders mv old.txt new.txt (renames or moves file)
rm [file]	Remove (delete) files rm temp.txt (deletes temp.txt)

3. VIEW FILE CONTENT

cat [file]	Display entire file cat notes.txt (shows full content)
less [file]	View file page by page less largefile.log (scroll with arrows / q to quit)
head [file]	Show first 10 lines head file.log (first 10 lines)
tail [file]	Show last 10 lines tail file.log (last 10 lines)

4. ABSOLUTE VS RELATIVE PATHS

ABSOLUTE PATH (Full path from root)



Example:

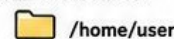
/home/user/docs/file.txt

✓ Starts from the root /

✓ Always points to the correct location

RELATIVE PATH (Based on current location)

Current location:



Example:

docs/file.txt

✓ Does not start with /

✓ Depends on where you are

5. COMBINING COMMANDS

Use ; to run commands one after another	mkdir test ; cd test ; touch file.txt
Use && to run next command only if the first succeeds	mkdir test && cd test
Use to run next command if the first fails	mkdir test echo "Failed to create"
Use to send output of one command to another	ls -l less
Use > to redirect output to a file (overwrite)	ls > output.txt
Use >> to redirect output to a file (append)	echo "Hello" >> notes.txt

6. READING COMMAND OUTPUT

 ls -l	Lists files in long format.
 ls -l less	Sends output to less so you can scroll.
 cat file.txt head -n 5	Shows first 5 lines of a file.
 grep "error" app.log	Searches for the word "error" in the file.

7. BEGINNER PRACTICE EXERCISE

1 SHOW WHERE YOU ARE pwd	→	2 LIST FILES AND FOLDERS ls	→	3 CREATE A FOLDER mkdir practice	→	4 GO INTO THE FOLDER cd practice	→	5 CREATE A FILE touch hello.txt	→	6 WRITE SOMETHING IN THE FILE echo "Hello" > hello.txt
7 VIEW FILE CONTENT cat hello.txt	→	8 COPY THE FILE cp hello.txt hello_copy.txt	→	9 RENAME THE FILE mv hello_copy.txt renamed.txt	→	10 LIST FILES TO SEE CHANGES ls -l	→	11 GO BACK ONE DIRECTORY cd ..	→	12 REMOVE THE FOLDER rm -r practice

3. CREATING AND RUNNING SHELL SCRIPTS



TURN COMMANDS INTO AUTOMATION

1. CREATING .SH FILES

- Create a new file with .sh extension.
- Use any text editor: nano, vim, code, etc.
- Write your commands inside.

Example:

```
nano myscript.sh
```

2. THE SHEBANG #!/bin/bash

- The first line of your script should be the shebang. It tells the system to use Bash to run the script.

Example:

```
#!/bin/bash
```

Always use this at the top!

3. MAKE IT EXECUTABLE chmod +x

- By default, scripts are not executable.
- Give execute permission.

Example:

```
chmod +x myscript.sh
```

4. RUN YOUR SCRIPT

Method 1 (Recommended)

```
./myscript.sh
```

Method 2

```
bash myscript.sh
```



Tip: Use ./ when running from the current directory.

5. FILE PERMISSIONS

Permission	Symbol	Number	Meaning
Read	r	4	Read the file
Write	w	2	Modify the file
Execute	x	1	Run the file

Example Permission: `-rwxr-xr--`

Owner Group Others

Check permissions with: `ls -l myscript.sh`



6. COMMON "PERMISSION DENIED" ERRORS

- Script is not executable.
- Wrong file permissions.
- Trying to run without ./
- Script has Windows line endings (CRLF).
- Script saved with wrong ownership.



Fix:

```
chmod +x myscript.sh
```

7. DEBUGGING A SCRIPT THAT WILL NOT EXECUTE

Problem	Why It Happens	How to Fix
 Permission denied	File is not executable or wrong permissions.	<pre>chmod +x myscript.sh</pre> <pre>ls -l myscript.sh</pre>
 Command not found	Shebang missing or wrong, or command doesn't exist.	Check first line: <code>#!/bin/bash</code> Use full path to command if needed.
 No such file or directory	Wrong path or filename.	Check spelling and path: <pre>ls -l pwd</pre>
 ^M: bad interpreter	Windows line endings (CRLF) in the script.	Fix line endings: <pre>sed -i 's/\r\$/' myscript.sh</pre>
 Script runs but nothing happens	No execute commands or logic errors.	Add <code>set -x</code> at top for debug. <pre>bash -x myscript.sh</pre>

8. BEST PRACTICES

- ✓ Always start with `#!/bin/bash`
- ✓ Make your script executable.
- ✓ Use full paths for important commands.
- ✓ Add comments to explain your code.
- ✓ Test your script step by step.
- ✓ Use `set -e` to exit on error.
- ✓ Keep scripts simple and readable.



Remember:
Great scripts start with good habits!



4. VARIABLES AND ENVIRONMENT VARIABLES



— STORE DATA. REUSE. AUTOMATE. —

1. CREATING VARIABLES

No spaces around =

```
NAME=DevOps  
COUNT=5
```

No spaces around = sign.

2. READING VARIABLES

Use \$ to read a variable.

```
echo $NAME
```

→ Output: **DevOps**

3. QUOTING VARIABLES

- Use quotes to handle spaces.
- Prevents word splitting.
- Prevents filename expansion.

```
echo "$NAME"  
echo '$NAME'
```

4. COMMAND SUBSTITUTION

Store the output of a command.

```
DATE=$(date +%Y-%m-%d)  
echo $DATE
```

→ Output: **2025-05-18**

5. EXPORT VARIABLES

- Makes a variable available to child processes.
- Use export.

```
export PROJECT=MyApp  
echo $PROJECT
```

→ Output: **MyApp**

6. ENVIRONMENT VARIABLES

Set by the system.
Available in every shell.

```
echo $HOME  
echo $USER  
echo $SHELL  
echo $LANG
```

7. COMMON ENVIRONMENT VARIABLES



HOME

Your home directory.

```
echo $HOME  
→ /home/user
```



USER

Current logged-in user.

```
echo $USER  
→ user
```



PATH

Directories where commands are searched.

```
echo $PATH  
→ /usr/bin:/bin:/usr/local/bin
```

8. USING VARIABLES IN SCRIPTS

Make your scripts flexible and powerful.

```
#!/bin/bash  
APP_NAME="MyApp"  
LOG_DIR="$HOME/logs"  
mkdir -p "$LOG_DIR"  
echo "Starting $APP_NAME..."  
echo "Logs will be saved in $LOG_DIR"
```

9. BEST PRACTICES

- ✓ Always use quotes when in doubt.
- ✓ Use meaningful variable names.
- ✓ Export only what you need.
- ✓ Keep variables uppercase for constants.
- ✓ Use command substitution \$() often.
- ✓ Test your scripts with different values.





5. READING USER INPUT AND SCRIPT ARGUMENTS



GET INPUT. USE ARGUMENTS. BUILD SMARTER SCRIPTS.

1. READ: GET USER INPUT

Use read to get input from the user.

```
read -p "Enter your name: " NAME
echo "Hello, $NAME"
```



-p shows a prompt.
The value is stored in the variable.

2. SCRIPT ARGUMENTS

Pass values to a script when you run it.

SYMBOL	MEANING	EXAMPLE
\$0	Script name	./deploy.sh
\$1	First argument	dev
\$2	Second argument	us-east-1
\$3	Third argument	app1
\$#	Number of arguments	3
\$@	All arguments	"\$@"

3. PASSING ARGUMENTS

Run your script with values.

```
./deploy.sh dev us-east-1 app1
```

Inside the script:

```
$1 = dev
$2 = us-east-1
$3 = app1
```



4. VALIDATING ARGUMENTS

Always check if arguments are provided.

```
if [ $# -lt 3 ]; then
  echo "Usage: $0 <env> <region> <app>"
  exit 1
fi
```



Prevents errors and confusion.

5. COMMON PRACTICES

- ✓ Always validate arguments.
- ✓ Use meaningful names.
- ✓ Quote "\$@" when using all args.
- ✓ Give helpful usage messages.
- ✓ Exit with code 1 on error.

6. EXAMPLE: SIMPLE REUSABLE DEVOPS SCRIPT

```
#!/bin/bash
# Check arguments
if [ $# -lt 3 ]; then
  echo "Usage: $0 <env> <region> <app>"
  exit 1
fi
# Assign arguments to variables
ENV=$1
REGION=$2
APP=$3
# Use variables
echo "Environment: $ENV"
echo "Region: $REGION"
echo "Application: $APP"
echo "Deploying $APP to $ENV in $REGION..."
```



RUN EXAMPLE

```
$ ./deploy.sh dev us-east-1 app1
Environment: dev
Region: us-east-1
Application: app1
Deploying app1 to dev in us-east-1...
```

WHAT THIS SCRIPT DOES

- 1 Checks if 3 arguments are provided.
- 2 Assigns values to variables.
- 3 Prints the values.
- 4 Ready to add real DevOps tasks.



TIP: Good scripts take input, validate it, and do useful work.
That's the foundation of automation!





6. EXIT CODES AND COMMAND SUCCESS



EVERY COMMAND RETURNS A STATUS.

1. WHAT EXIT CODES MEAN



After any command runs, the system returns a number called an exit code. It tells you if the command succeeded or failed.

`echo $?` Shows the exit code of the last command.

2. BASIC EXIT CODE RULES

0

EXIT CODE 0
Success

Command completed successfully.

1+

NON-ZERO EXIT CODE
Failure

Something went wrong. Check the error.

3. EXAMPLES

`ls /home` → `echo $?`
Output: 0

`ls /notfoundfolder` → `echo $?`
Output: 2

`cat file.txt` → `echo $?`
Output: 0 or 1+

4. THE exit COMMAND

Use `exit` to end a script and return a code.

```
if [ ! -f file.txt ]; then
  echo "File not found!"
  exit 1
fi
```

`exit 0` = success
`exit 1` = failure



Ends the script with a status.

5. TESTING COMMAND SUCCESS

Use exit codes in conditions.

```
ls /home
if [ $? -eq 0 ]; then
  echo "Success"
else
  echo "Failed"
fi
```

✓ `$? -eq 0`
Success

✗ `$? -ne 0`
Failure

6. WHY CI/CD PIPELINES DEPEND ON EXIT CODES



- Each step returns an exit code.
- CI/CD tools stop the pipeline on failure.
- Exit code 0 = pipeline continues.
- Exit code 1+ = pipeline fails.
- Ensures reliable and consistent deployments.

7. PREVENTING SILENT AUTOMATION FAILURES



Always check exit codes.

Don't ignore command results.



Use `set -e` to exit on error.

Stops the script on first failure.



Log errors clearly.

Makes debugging much easier.



Validate inputs before running.

Bad inputs cause unexpected failures.



Test your scripts regularly.

Catches issues early.



TIP: Never ignore exit codes. Check them, handle them, and your scripts will be reliable and safe.



YOUTUBE



GITHUB



LINKEDIN

VERIQTA



X



INSTAGRAM



TELEGRAM



7. CONDITIONAL STATEMENTS

MAKE SMART DECISIONS IN YOUR SCRIPTS.



1. BASIC SYNTAX

```
if [ condition ]; then
    commands
elif [ condition ]; then
    commands
else
    commands
fi
```

if - Start condition
then - Run if true
elif - Else if another condition
else - Run if all false
fi - End condition

2. NUMERIC COMPARISONS

-eq	Equal	if [\$a -eq \$b]
-ne	Not equal	if [\$a -ne \$b]
-gt	Greater than	if [\$a -gt \$b]
-lt	Less than	if [\$a -lt \$b]
-ge	Greater or equal	if [\$a -ge \$b]
-le	Less or equal	if [\$a -le \$b]

3. STRING COMPARISONS

=	Equal	if ["\$a" = "\$b"]
!=	Not equal	if ["\$a" != "\$b"]
-z	String is empty	if [-z "\$a"]
-n	String not empty	if [-n "\$a"]



Always quote strings!
Prevents errors with spaces.

4. FILE TESTS

-f	File exists	if [-f file.txt]
-d	Directory exists	if [-d /home/user]
-e	File or directory exists	if [-e /tmp/log]
-r	Readable	if [-r file.txt]
-w	Writable	if [-w file.txt]
-x	Executable	if [-x script.sh]
-s	File is not empty	if [-s file.txt]

5. CHECKING SERVICES, FILES, OR DIRECTORIES



Check if a file exists

```
if [ -f /etc/passwd ]; then
    echo "File exists"
else
    echo "File not found"
fi
```



Check if a directory exists

```
if [ -d /var/log ]; then
    echo "Directory exists"
else
    echo "Directory not found"
fi
```



Check if a service is running

```
if systemctl is-active --quiet nginx; then
    echo "Nginx is running"
else
    echo "Nginx is not running"
fi
```

6. PRACTICAL EXAMPLE

Check disk space and act based on the result.

```
USAGE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
if [ "$USAGE" -gt 80 ]; then
    echo "Disk usage is high: ${USAGE}%"
elif [ "$USAGE" -gt 60 ]; then
    echo "Disk usage is moderate: ${USAGE}%"
else
    echo "Disk usage is normal: ${USAGE}%"
fi
```

7. QUICK REFERENCE

if	Start condition	✓ Use spaces inside [].
then	Run if true	✓ Quote strings.
elif	Else if	✓ Use numeric operators without quotes.
else	Run if all false	✓ Test before you act.
fi	End condition	✓ Make scripts smart and reliable.



TIP: Conditions make your scripts intelligent.
Always test, validate, and handle every situation.



YOUTUBE



GITHUB



LINKEDIN



X



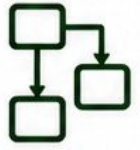
INSTAGRAM



TELEGRAM



8. LOGICAL OPERATORS AND MULTIPLE CONDITIONS



COMBINE TESTS. MAKE SMARTER DECISIONS.

1. LOGICAL OPERATORS

OPERATOR	MEANING	EXAMPLE	DESCRIPTION
&&	AND	<pre>if [\$a -gt 5] && [\$b -lt 10]</pre>	Both conditions must be true.
	OR	<pre>if [\$a -gt 5] [\$b -lt 10]</pre>	At least one condition must be true.
!	NOT	<pre>if ! [-f file.txt]</pre>	Reverses the result.

2. SHORT-CIRCUIT EXECUTION

- &&** If the first condition is false, the second condition is **NOT** executed.
- ||** If the first condition is true, the second condition is **NOT** executed.
- !** Flips true to false, and false to true.
-  Saves time and prevents unnecessary checks.

3. AND CONDITIONS (&&)

Both conditions must be true.

```
if [ -f app.sh ] && [ -x app.sh ]; then  
    echo "File exists and is executable"  
else  
    echo "Check failed"  
fi
```

- ✓ Use for strict requirements.

4. OR CONDITIONS (||)

At least one condition must be true.

```
if [ -f app.sh ] || [ -f app.py ]; then  
    echo "Script found"  
else  
    echo "No script found"  
fi
```

- ✓ Use for alternatives.

5. NOT CONDITION (!)

Reverse a condition.

```
if [ ! -f config.env ]; then  
    echo "Config file missing"  
else  
    echo "Config file exists"  
fi
```

- ✓ Use to detect missing items.

6. COMBINING MULTIPLE TESTS

You can mix &&, ||, and ! for powerful checks.

EXAMPLE 1: MIX AND & OR

```
if [ -f deploy.sh ] && [ -x deploy.sh ] ||  
[ -f deploy.py ]; then  
    echo "Ready to deploy"  
else  
    echo "Deployment script missing"  
fi
```

- ✓ Executable deploy.sh OR deploy.py exists.

EXAMPLE 2: NEGATION WITH AND

```
if [ ! -f .lock ] && [ -d /var/log ]; then  
    echo "Safe to proceed"  
else  
    echo "Lock file exists or log  
    directory missing"  
fi
```

- ✓ No lock file AND directory exists.

EXAMPLE 3: COMPLEX COMBINATION

```
if [ -f app.conf ] && [ -r app.conf ] &&  
! [ -z "$1" ]; then  
    echo "All checks passed"  
else  
    echo "Validation failed"  
fi
```

- ✓ File exists, readable AND first argument is not empty.

7. BUILDING SAFER DEPLOYMENT CHECKS

Example: Before deploying, validate everything.

```
if [ -f app.tar.gz ] && [ -s app.tar.gz ] && [ -x /usr/bin/docker ] &&  
! systemctl is-active --quiet app; then  
    echo "All checks passed. Starting deployment..."  
    # Your deployment commands here  
else  
    echo "Deployment aborted. Check requirements."  
    exit 1  
fi
```

- ✓ File exists AND not empty
- ✓ Docker is installed
- ✓ App service is not running
- ✓ Proceed only when everything is correct
- ✓ Prevents failed or risky deployments



SAFE
DEPLOYMENT
STARTS HERE!



TIP: Combine checks, validate early, and fail fast.
Smart conditions = reliable automation.





9. LOOPS FOR AUTOMATION



REPEAT TASKS. SAVE TIME. ELIMINATE MANUAL WORK.

1. FOR LOOPS

Loop through a list of items.

```
for item in a b c; do
  echo "Item: $item"
done
```



→ Output:

Item: a Item: b Item: c

2. WHILE LOOPS

Repeat while a condition is true.

```
count=1
while [ $count -le 5 ]; do
  echo "Count: $count"
  count=$((count + 1))
done
```



→ Output: Count: 1 2 3 4 5

3. LOOPING THROUGH FILES

Process each file in a directory.

```
for file in *.txt; do
  echo "File: $file"
done
```



Example: Prints all .txt files in the current directory.

4. LOOPING THROUGH SERVERS

Run commands on multiple servers.

```
for server in server1 server2 server3; do
  echo "Checking $server..."
  ssh $server uptime
done
```



→ Example: Check uptime on many servers

5. BREAK

Exit the loop immediately.

```
for i in {1..5}; do
  if [ $i -eq 3 ]; then
    break
  fi
  echo "Number: $i"
done
```



STOP
THE LOOP

Output: Number: 1 2

6. CONTINUE

Skip the rest of the loop and continue.

```
for i in {1..5}; do
  if [ $i -eq 3 ]; then
    continue
  fi
  echo "Number: $i"
done
```



SKIP AND
CONTINUE

Output: Number: 1 2 4 5

7. AVOIDING INFINITE LOOPS

Always update your condition.

- ⚠ Always change the variable inside the loop.
- ⚠ Use clear exit conditions.
- ⚠ Test your loops with small values first.
- ⚠ Use break when needed.

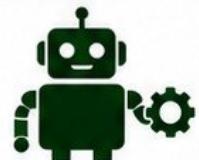


BE CAREFUL

8. AUTOMATING REPETITIVE TASKS

Loops help you automate daily tasks.

- ✓ File management
- ✓ Server monitoring
- ✓ User management
- ✓ Log analysis
- ✓ Backups and cleanups
- ✓ System updates



AUTOMATE
EVERYTHING



**KEY
TAKEAWAY**

Loops save time, reduce errors, and make your scripts powerful.



REPEAT
SMARTER



SAVE TIME
EVERY DAY



10. FUNCTIONS AND REUSABLE SCRIPTS



VERIQTA

— WRITE ONCE. USE MANY TIMES. STAY ORGANIZED. —

VERIQTA

1. CREATING FUNCTIONS

Define a function using a name and () block.

```
greet() {
  echo "Hello, DevOps Engineer!"
}
```



2. CALLING FUNCTIONS

Call the function by its name.

```
greet
```



→ Output: Hello, DevOps Engineer!

3. FUNCTION ARGUMENTS

Pass values to functions.

```
greet_user() {
  echo "Hello, $1!"
}
greet_user "Ann"
```



→ Output: Hello, Ann!

4. LOCAL VARIABLES

Keep variables private to the function.

```
calculate() {
  local num=10
  echo "Inside function: $num"
}
calculate
echo "Outside: $num"
```



**LOCAL
MEANS
PRIVATE**

→ Output:
Inside function: 10 Outside:

5. RETURN STATUS

Functions return an exit status.

```
check_disk() {
  if [ $1 -gt 80 ]; then
    return 1
  else
    return 0
  fi
}
check_disk 90
echo "Status: $?"
```



0 = SUCCESS
1 = FAILURE

→ Output: Status: 1

6. BREAK LARGE SCRIPTS INTO FUNCTIONS

Make scripts smaller, cleaner, and easier to read.

```
check_system()
```

```
backup_files()
```

```
deploy_app()
```

```
send_alert()
```

```
cleanup()
```

```
print_summary()
```



MAIN SCRIPT



CALL
FUNCTIONS



DONE



SUCCESS

7. REUSABLE AUTOMATION PATTERNS

- ✓ Health checks
- ✓ Backups
- ✓ Deployments
- ✓ Log rotations
- ✓ User management
- ✓ System cleanups



8. KEEP SCRIPTS MAINTAINABLE

- ✓ Use meaningful function names.
- ✓ Keep functions small and focused.
- ✓ Add comments to explain purpose.
- ✓ Test each function before using it.
- ✓ Reuse functions across scripts.



**CLEAN CODE
SAVES TIME
AND HEADACHE**



KEY

TAKEAWAY

Functions make your scripts powerful, reusable, and easy to maintain.



**BUILD ONCE
USE MANY TIMES**



**WRITE SMARTER
AUTOMATE BETTER**

VERIQTA



11. WORKING WITH FILES AND DIRECTORIES



VERIQTA

— ORGANIZE. PROTECT. AUTOMATE. —

VERIQTA

1. CHECKING WHETHER FILES EXIST

Use `-f` for files, `-d` for directories.

```
if [ -f file.txt ]; then
  echo "File exists"
fi
if [ -d mydir ]; then
  echo "Directory exists"
fi
```



2. CREATING DIRECTORIES SAFELY

Create only if it does not exist.

```
mkdir -p /home/user/data/logs
```



`-p` creates parent directories if needed.

3. COPYING AND MOVING FILES

Copy files or directories.

```
cp file.txt /backup/
cp -r mydir /backup/
```



Move or rename files.

```
mv old.txt new.txt
mv file.txt /archive/
```



4. SEARCHING FOR FILES

Find files by name.

```
find /home/user -name "*.log"
```



Search by type.

```
find /var/log -type f -name "*.log"
```

5. FILE PERMISSIONS

View permissions.

```
ls -l file.txt
```

Change permissions.

```
chmod 644 file.txt
chmod +x script.sh
```



6. DISK USAGE CHECKS

Check disk usage.

```
df -h
```

Check folder size.

```
du -sh /home/user/data
```



7. CLEANING TEMPORARY FILES

Remove files older than 7 days.

```
find /tmp -type f -mtime +7 -delete
```

Remove all files in a directory.

```
rm -rf /tmp/myapp/*
```



8. BUILDING A SIMPLE BACKUP SCRIPT

Example: Backup a folder.

```
#!/bin/bash
SOURCE="/home/user/data"
DEST="/backup"
DATE=$(date +%F)
tar -czf "$DEST/backup-$DATE.tar.gz" "$SOURCE"
echo "Backup completed: backup-$DATE.tar.gz"
```



KEY TAKEAWAY

Good file management keeps your systems clean, safe, and reliable.



PROTECT YOUR DATA



AUTOMATE DAILY TASKS



STAY ORGANIZED AND EFFICIENT



12. TEXT PROCESSING FOR DEVOPS



VERIQTA

— EXTRACT. FILTER. ANALYZE. AUTOMATE. —

VERIQTA

1 grep – Search text patterns

Search for a pattern in files or output.

```
grep "ERROR" app.log
```



2 cut – Extract columns

Extract parts of each line.

```
cut -d',' -f1 users.csv
```



3 sort – Sort lines

Sort lines alphabetically or numerically.

```
sort names.txt
```



4 uniq – Remove duplicates

Remove or count duplicate lines.

```
sort names.txt | uniq -c
```



5 wc – Count lines, words, bytes

Count lines, words, and characters.

```
wc -l file.txt
```



6 awk – Pattern scanning and processing

Extract and manipulate data.

```
awk '{print $1, $3}' data.txt
```



7 sed – Stream editor

Find and replace text.

```
sed 's/old/new/g' file.txt
```



8 Pipes (|) – Connect commands

Send output of one command to another.

```
cat app.log | grep ERROR | wc -l
```



9. EXTRACTING USEFUL INFORMATION FROM COMMAND OUTPUT

Turn raw output into useful data.

Example: List top 5 largest files

```
du -h | sort -h | tail -n 5
```



Example: Count unique IP addresses

```
cut -d' ' -f1 access.log | sort | uniq -c | sort -nr
```

Example: Show running processes summary

```
ps aux | awk '{print $1}' | sort | uniq -c | sort -nr | head
```

10. SEARCHING APPLICATION LOGS

Find errors, warnings, and important events fast.

Example: Find all ERROR lines

```
grep "ERROR" /var/log/app.log
```



Example: Find errors with line numbers

```
grep -n "ERROR" /var/log/app.log
```

Example: Count warnings

```
grep -c "WARNING" /var/log/app.log
```

11. COMMON COMBINATIONS

- ✓ `grep "ERROR" app.log | wc -l` Count errors in log
- ✓ `cat file.txt | sort | uniq` Unique sorted lines
- ✓ `ps aux | grep nginx` Find nginx processes
- ✓ `cat data.txt | awk '{print $2}' | sort | uniq -c` Count occurrences of column 2
- ✓ `grep -i "failed" *.log | sort | uniq -c` Case-insensitive search in all logs

12. BEST PRACTICES



Use the right tool for the job.



Filter early, filter often.



Combine commands with pipes.



Always verify your output.



Automate log analysis tasks.



YOUTUBE



GITHUB



LINKEDIN



X



INSTAGRAM



TELEGRAM



13. REDIRECTION, PIPES, AND LOGS



VERIQTA — CONTROL OUTPUT. CAPTURE LOGS. DEBUG SMARTER. — VERIQTA

1. STANDARD INPUT (STDIN)

Input from keyboard or file.

```
command < file.txt
sort < names.txt
```



2. STANDARD OUTPUT (STDOUT)

Normal command output.

```
ls -l
echo "Hello"
```



3. STANDARD ERROR (STDERR)

Error messages and warnings.

```
ls no_file
cat no_file.txt
```



4. OUTPUT REDIRECTION

Overwrite output to a file.

```
command > file.log
```



OVERWRITE FILE

5. APPEND REDIRECTION

Append output to a file.

```
command >> file.log
```



APPEND TO FILE

6. ERROR REDIRECTION

Redirect errors to a file.

```
command 2> error.log
```

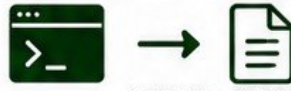


ERRORS ONLY

7. COMBINE STDOUT AND STDERR

Send both output and errors to same file.

```
command > output.log 2>&1
```



OUTPUT + ERRORS
TOGETHER

8. PIPES (|)

Send output of one command as input to another.

```
ps aux | grep nginx | wc -l
```



FILTER AND PROCESS DATA

9. SEND SCRIPT OUTPUT TO LOG FILES

Log everything for monitoring and audits.

```
#!/bin/bash
echo "Starting script..." > script.log
ls -l /var/log >> script.log
df -h >> script.log
echo "Script completed." >> script.log
```



10. CAPTURING ERRORS SEPARATELY

Keep success and errors in different files.

```
#!/bin/bash
command > success.log 2> error.log
echo "Done" >> success.log
```



11. COMMON EXAMPLES

- ✓ `ls -l > files.txt` → Save output to file (overwrite)
- ✓ `ls -l >> files.txt` → Append output to file
- ✓ `command 2> error.log` → Save errors to error.log
- ✓ `command > all.log 2>&1` → Save output and errors to all.log
- ✓ `command | grep "pattern"` → Filter output using grep
- ✓ `cat access.log | grep 404 | wc -l` → Count 404 errors in logs



BEST PRACTICES

- ✓ Log important outputs.
- ✓ Separate errors from normal output.
- ✓ Use append (>>) for long-running logs.
- ✓ Use pipes to analyze data.
- ✓ Always check your logs.
- ✓ Keep logs organized and rotate them.



14. PROCESSES AND SERVICES

— MONITOR. CONTROL. KEEP EVERYTHING RUNNING. —

1. PS – VIEW PROCESSES

Show running processes.

```
ps aux
ps -ef
```



Useful output columns:

- PID – Process ID
- USER – Owner
- %CPU – CPU usage
- %MEM – Memory usage
- COMMAND – Process name

2. PGREP – FIND PROCESSES

Find process IDs by name.

```
pgrep nginx
pgrep -f myapp
```



Used for scripting and automation.

3. KILL – STOP PROCESSES

Stop processes using PID.

```
kill 1234
kill -9 1234
```



- kill – graceful stop (SIGTERM)
- kill -9 – force stop (SIGKILL)

⚠ Use kill -9 only when needed.

4. SYSTEMCTL – MANAGE SERVICES

Control system services (Linux).

```
systemctl start nginx
systemctl stop nginx
systemctl restart nginx
systemctl enable nginx
```



Works with systemd services.

5. CHECK IF PROCESS IS RUNNING

Check if a process is running.

```
pgrep nginx >/dev/null
if [ $? -eq 0 ]; then
    echo "Running"
else
    echo "Not running"
fi
```



Exit code 0 = running

6. CHECK SERVICE STATUS

Check status of a service.

```
systemctl status nginx
```

Common states:

- active (running)
- inactive (dead)
- failed



7. RESTART FAILED SERVICES

Restart a service.

```
systemctl restart nginx
```

Auto-restart on boot (optional):

```
systemctl enable nginx
```

Check again:

```
systemctl status nginx
```



8. PROCESS INFORMATION

Get more details about a process.

```
ps -p 1234 -o pid,user,cmd,%mem,%cpu
```

Show parent process:

```
ps -o pid,ppid,cmd -p 1234
```



9. BASIC SERVICE HEALTH CHECK SCRIPT

Example: Check and restart service.

```
#!/bin/bash
SERVICE=nginx
pgrep $SERVICE >/dev/null
if [ $? -eq 0 ]; then
    echo "$SERVICE is running"
else
    echo "$SERVICE is down. Restarting..."
    systemctl restart $SERVICE
fi
```

10. COMMON COMMANDS QUICK REFERENCE

- | | |
|---------------------------|---------------------------|
| ✓ ps aux | → List all processes |
| ✓ pgrep <name> | → Find process ID |
| ✓ kill <PID> | → Stop process gracefully |
| ✓ kill -9 <PID> | → Force stop process |
| ✓ systemctl status <svc> | → Check service status |
| ✓ systemctl restart <svc> | → Restart service |
| ✓ systemctl enable <svc> | → Auto-start on boot |
| ✓ journalctl -u <svc> | → View service logs |



BEST PRACTICES

- ✓ Always check before you kill.
- ✓ Use meaningful process names.
- ✓ Monitor important services.
- ✓ Automate health checks.
- ✓ Log issues for faster troubleshooting.
- ✓ Restart only when necessary.
- ✓ Keep scripts simple and readable.



KEY TAKEAWAY

Know your processes.
Control your services.
Keep your systems reliable.



MONITOR
PROCESSES



MANAGE
SERVICES



KEEP SYSTEMS
HEALTHY



15. NETWORKING WITH SHELL SCRIPTS

TEST. MONITOR. ENSURE AVAILABILITY.

1. CURL – SEND HTTP REQUESTS

Send HTTP requests to endpoints.

```
curl https://example.com
curl -I https://example.com
curl -s -o /dev/null -w "%{http_code}"
https://example.com
```

-I Fetch headers only
-s Silent mode
-o /dev/null Discard body
-w "%{http_code}" Show status code



2. PING – TEST CONNECTIVITY

Test whether a host is reachable.

```
ping -c 4 google.com
ping -c 4 8.8.8.8
```



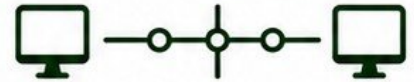
Common use:

- ✓ Check network connectivity
- ✓ Troubleshoot reachability issues
- ✓ Measure response time

3. SS – CHECK CONNECTIONS

Show network sockets and connections.

```
ss -tln # Listening ports
ss -tnp # All connections
ss -ltnp # Listening with PID
```



Useful to:

- ✓ Check open ports
- ✓ See active connections
- ✓ Diagnose network issues

4. HOSTNAME – SHOW SYSTEM NAME

Get or set the system hostname.

```
hostname # Show hostname
hostname -I # Show IP addresses
hostnamectl # Detailed info
```



Server: web01
IP: 192.168.1.10

Use cases:

- ✓ Identify servers
- ✓ Log server identity
- ✓ Script environment awareness

5. DNS CHECKS – RESOLVE NAMES

Check DNS resolution.

```
nslookup example.com
dig example.com
host example.com
```



Useful to:

- ✓ Verify DNS records
- ✓ Troubleshoot name resolution
- ✓ Check DNS propagation

6. HTTP STATUS CODES

Check HTTP response codes.

```
curl -o /dev/null -s -w "%{http_code}"
https://example.com
```

200	OK – Success
301/302	Redirect
404	Not Found
500	Server Error

Always check status codes, not just ping!

7. TEST ENDPOINT AVAILABILITY

Quickly test if an endpoint is up.

```
curl -s -o /dev/null -w "%{http_code}"
https://example.com
```

Returns:

- 200–399 = Available
- 400–599 = Not Available
- 000 = No Response



8. CHECK APPLICATION HEALTH

Check if application health endpoint is OK.

```
curl -s https://example.com/health
curl -s -f https://example.com/health
&& echo "Healthy" || echo "Unhealthy"
```



Best practice:

Expose a /health endpoint in apps.

9. SIMPLE WEBSITE AVAILABILITY SCRIPT

Example: Check website availability.

```
#!/bin/bash
URL="https://example.com"
STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$URL")
if [ "$STATUS" = "200" ]; then
  echo "[OK] $URL is UP (200)"
else
  echo "[DOWN] $URL is DOWN (Code: $STATUS)"
fi
```



CHECK URL



GET STATUS

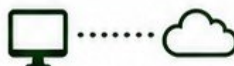


REPORT RESULT

10. COMMON NETWORKING COMMANDS QUICK REFERENCE

```
curl URL
ping HOST
ss -tln
hostname
nslookup DOMAIN
dig DOMAIN
host DOMAIN
curl -I URL
curl -o /dev/null -w
"%{http_code}" URL
```

Send HTTP request
Test connectivity
Show listening ports
Show hostname
DNS lookup (basic)
DNS lookup (advanced)
DNS lookup (simple)
Fetch HTTP headers
Get HTTP status code



- ✓ Test
- ✓ Monitor
- ✓ Ensure Availability

BEST PRACTICES

- ✓ Always check HTTP status codes.
- ✓ Use timeouts to avoid hanging scripts.
- ✓ Check both connectivity and application health.
- ✓ Log results for monitoring and alerting.
- ✓ Use meaningful alerts when a service is down.
- ✓ Test from the same network as users.
- ✓ Keep scripts simple, clear, and maintainable.



KEY TAKEAWAY

Automate your network checks.
Detect issues early.
Keep your systems and services reliable.



CHECK NETWORK



VERIFY HEALTH

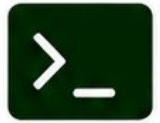


ENSURE UPTIME



VERIQTA

16. ERROR HANDLING AND SAFER SCRIPTS



VERIQTA

WRITE PREDICTABLE. RELIABLE. SAFE AUTOMATION.

1. WHY SCRIPTS FAIL SILENTLY

By default, Bash continues even when commands fail.

```
rm file.txt
echo "Deleted"
echo "Script finished"
```

Problem:

If rm fails, the script keeps running and hides the error.

2. SET -e : EXIT ON ERROR

Exit immediately if any command fails.

```
set -e
rm file.txt
echo "This runs only if rm succeeds"
```

Benefit:

Stops the script when a command fails.

3. SET -u : ERROR ON UNSET VARIABLES

Exit if you use a variable that is not set.

```
set -u
echo $UNSET_VAR
```

Benefit:

Catches typos and missing variables early.

VERIQTA

4. SET -o PIPEFAIL : CATCH PIPE ERRORS

Detect failures inside pipelines.

```
set -o pipefail
cat file.txt | grep "ERROR" | wc -l
```

Benefit:

Without pipefail, only the last command's status is checked.

5. SET -euo PIPEFAIL : STRONG & SAFE MODE

Best practice for safer scripts.

```
set -euo pipefail
```

Benefits:

- Exit on any command error
- Error on unset variables
- Catch pipeline failures



WRITE SAFER.
SLEEP BETTER.

VERIQTA

6. VALIDATING INPUTS



Always verify inputs before using them.

```
if [ -z "$1" ]; then
  echo "Usage: $0 <url>"
  exit 1
fi
```

Benefit:

Prevents errors and makes scripts user-friendly.

7. CHECKING DEPENDENCIES



Ensure required commands exist before running.

```
if ! command -v curl >/dev/null 2>&1; then
  echo "curl not found!"
  exit 1
fi
```

Benefit:

Avoids failures due to missing tools.

8. HANDLING FAILED COMMANDS



Check command results and handle failures.

```
if ! rm file.txt; then
  echo "Failed to remove file.txt"
  exit 1
fi
```

Benefit:

React to failures and prevent bad outcomes.

9. CLEANING UP AFTER FAILURES



Use traps to clean up on exit or error.

```
cleanup() {
  echo "Cleaning up..."
  rm -f temp.log
}
trap cleanup EXIT
```

Benefit:

Keeps systems clean even when scripts fail.

10. WRITING PREDICTABLE AUTOMATION



- ✓ Fail fast
- ✓ Validate inputs
- ✓ Check dependencies
- ✓ Handle errors
- ✓ Clean up properly
- ✓ Log what happens
- ✓ Exit with the right status code

Result:
Reliable and predictable automation every time.

VERIQTA

EXAMPLE: SAFE SCRIPT TEMPLATE

```
#!/bin/bash
set -euo pipefail
# Check input
if [ -z "$1" ]; then
  echo "Usage: $0 <url>"
  exit 1
fi
# Check dependency
command -v curl >/dev/null 2>&1 || { echo "curl not found!"; exit 1; }
# Function to cleanup
cleanup() { echo "Cleaning up..."; rm -f temp.log; }
trap cleanup EXIT
# Main logic
URL="$1"
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" "$URL")
if [ "$HTTP_CODE" -ge 200 ] && [ "$HTTP_CODE" -lt 400 ]; then
  echo "[URL] is UP (HTTP $HTTP_CODE)"
else
  echo "[URL] is DOWN (HTTP $HTTP_CODE)"
fi
exit 1
```

- **Safe Mode**
set -euo pipefail makes the script strict and safe.
- **Validate Input**
Ensure the user provides the required input.
- **Check Dependency**
Stop early if a required tool is missing.
- **Cleanup on Exit**
Cleanup runs automatically on success or failure.
- **Main Logic**
Do the work and check results.
- **Exit Status**
Exit 0 for success, non-zero for failure.

VERIQTA



KEY TAKEAWAY

Safe scripts = Less downtime, fewer bugs, and happier users.
Always handle errors. Always clean up. Always be predictable.

VERIQTA



17. DEBUGGING SHELL SCRIPTS

FIND ISSUES. UNDERSTAND CAUSES. FIX FAST.



1. READING SHELL ERRORS

Errors tell you what went wrong. Read them carefully.

```
./script.sh: line 10:
mycmd: command not found
./script.sh: line 15:
[: : integer expression
expected
Permission denied
No such file or directory
```

TIP: The error message always gives a clue.

2. DEBUG MODE WITH bash -x

Run your script with bash -x to see each command.

```
bash -x script.sh
```

Example output:

```
+ echo "Starting"
Starting
+ VAR=hello
+ echo $VAR
hello
+ ls /not/exist
ls: cannot access '/not/exist':
No such file or directory
```

3. DEBUG MODE INSIDE SCRIPT

Use set -x to enable debug inside the script.

```
set -x
# Your commands here
set +x # Turn off debug
```

BENEFIT:
Shows each command as it runs.
Very helpful to trace issues.

4. COMMON PROBLEMS AND HOW TO FIND THEM

A. SYNTAX ERRORS

Missing symbols or wrong structure.

```
if [ "${VAR}" = "test"
echo "Missing fi"
# Error here
```

Fix:
Check brackets, quotes, then/fi, do/done.

B. QUOTING PROBLEMS

Wrong quotes break the script.

```
echo "Hello $USER" # Good
echo 'Hello $USER' # Literal
echo "He said "hi"" # Bad
```

Fix:
Use double quotes for variables.
Escape inner quotes.

C. MISSING VARIABLES

Using variables that are not set.

```
echo $NAME
./script.sh: line 5:
NAME: unbound variable
```

Fix:
Define the variable or use default value: \${NAME:-John}

D. PERMISSION ERRORS

Script or files are not executable.

```
./script.sh: line 3:
./other.sh: Permission denied
```

Fix:
Use chmod +x script.sh and check file permissions.

E. PATH PROBLEMS

Command not found.

```
mycmd: command not found
```

Fix:
Check PATH or use full path: /usr/bin/mycmd

F. FILE OR DIRECTORY MISSING

File or directory does not exist.

```
cat: file.txt: No such file
or directory
```

Fix:
Check the path and filename.
Use absolute path if needed.

5. DEBUGGING SCRIPTS STEP BY STEP

- | | | | |
|---|--|----------------------------|---|
| 1 | | Read the error | Understand what the error is saying. Look at the line number and message. |
| 2 | | Reproduce the issue | Run the script and confirm the error happens again. |
| 3 | | Run with bash -x | Use bash -x script.sh to see each command and where it fails. |
| 4 | | Check variables and quotes | Make sure variables are set and quoted correctly. |
| 5 | | Fix and test again | Make the change and run again. Repeat until the error is gone. |
| 6 | | Remove debug | Remove set -x or stop using bash -x after fixing. |



KEY TAKEAWAY

Good debugging saves hours.
Read errors. Use tools. Fix the root cause.



18. SCHEDULING SCRIPTS WITH CRON



VERIQTA

AUTOMATE TASKS. SAVE TIME. STAY RELIABLE.

1. WHAT CRON DOES



- Cron runs commands or scripts automatically at scheduled times.
- Perfect for backups, cleanup, monitoring, reports, and more.

2. UNDERSTANDING CRONTAB



- Crontab is your personal cron schedule.
- Edit with:

```
crontab -e
```

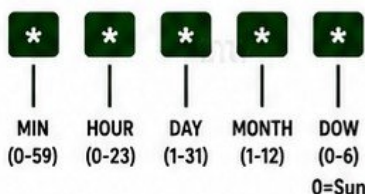
- List jobs:

```
crontab -l
```

- Remove all jobs:

```
crontab -r
```

3. CRON TIMING SYNTAX



EXAMPLES

- ✓ * * * * * Every minute
- ✓ 0 * * * * Every hour at :00
- ✓ 0 2 * * * Every day at 2:00 AM
- ✓ 30 1 * * 1 Every Monday at 1:30 AM
- ✓ 0 3 1 * * 1st day of every month at 3 AM

4. RUNNING SCRIPTS AUTOMATICALLY



- Add your cron job with `crontab -e`
- Cron runs jobs in the background.
- No need to keep your terminal open.
- Great for hands-off automation.

5. ENVIRONMENT DIFFERENCES IN CRON



- Cron runs with a minimal environment.
- Your PATH, variables, and shell may be different.
- Use full paths in scripts.
- Set needed variables inside the script.

Example:

```
PATH=/usr/local/bin:/usr/bin:/bin
export PATH
```

6. LOGGING CRON OUTPUT



- Always log output for debugging.
- Redirect output and errors.

Example:

```
*/5 * * * * /path/to/script.sh \
>> /var/log/mycron.log 2>&1
```

7. COMMON CRON FAILURES



- ✗ Wrong timing syntax
- ✗ Script path incorrect
- ✗ Script not executable
- ✗ Missing environment variables
- ✗ No permissions to run script
- ✗ Output not logged
- ✗ Mail not configured
- ✗ Cron service not running

8. SCHEDULING BACKUP AND CLEANUP JOBS



EXAMPLE BACKUP JOB

```
0 2 * * * /backup.sh \
>> /var/log/backup.log 2>&1
```

EXAMPLE CLEANUP JOB

```
30 3 * * 0 /cleanup.sh \
>> /var/log/cleanup.log 2>&1
```

CRON SYNTAX QUICK REFERENCE

*	Any value	1-31	Day of month
,	Values list (1,2,3)	1-12	Month
-	Range (1-5)	0-6	Day of week (0=Sun)
/	Step values (*/10)	@daily	Same as 0 0 * * *
0-59	Minute	@weekly	Same as 0 0 * * 0
0-23	Hour	@reboot	Run once at startup

BEST PRACTICES

- ✓ Use full paths for commands and scripts.
- ✓ Log all cron outputs.
- ✓ Test scripts manually before scheduling.
- ✓ Keep cron jobs simple and reliable.
- ✓ Review your crontab regularly.



19. SHELL SCRIPTING IN DEVOPS WORKFLOWS

AUTOMATE. INTEGRATE. DEPLOY.

1. SHELL SCRIPTS IN CI/CD



- Automate build, test, and deploy.
- Run commands at every stage.
- Make pipelines reliable and repeatable.
- Handle setup, checks, and notifications.

VERIQTA

2. GITHUB ACTIONS



GitHub
Actions

- Use shell scripts in workflow steps.
- Run custom commands easily.
- Prepare environments.
- Build, test, and deploy apps.

```
- name: Run script
  run: ./deploy.sh
```

3. JENKINS



Jenkins

- Execute shell scripts in build steps.
- Automate jobs and deployments.
- Manage environments.
- Integrate with other tools.

```
#!/bin/bash
./build.sh && ./deploy.sh
```

4. DOCKER



- Build and push images.
- Prepare container environments.
- Clean up unused resources.
- Manage volumes and networks.

```
#!/bin/bash
docker build -t myapp .
docker push myapp:latest
```

5. KUBERNETES



kubernetes

- Automate kubectl commands.
- Deploy and update applications.
- Manage configs and secrets.
- Check pod and service status.

```
kubectl apply -f app.yaml
kubectl rollout status deploy/app
```

6. CLOUD VM INITIALIZATION



- Run scripts on first boot.
- Install packages and tools.
- Configure users and services.
- Set up security and updates.

```
#!/bin/bash
apt update && apt install -y nginx
systemctl enable nginx
```

7. DEPLOYMENT SCRIPTS



- Automate deployments.
- Backup before updates.
- Rollback on failure.
- Zero- or low-downtime.

```
./deploy.sh --env prod
./rollback.sh
```

8. HEALTH CHECKS



- Check app and services.
- Validate endpoints.
- Alert on failure.
- Keep systems reliable.

```
#!/bin/bash
curl -f http://localhost/
|| exit 1
```

9. ENVIRONMENT VALIDATION



- Verify required tools.
- Check variables and configs.
- Validate dependencies.
- Fail fast if something is wrong.

```
command -v docker >/dev/null
|| { echo "Docker not found";
  exit 1; }
```

10. WHERE SHELL SCRIPTING FITS IN DEVOPS



- Glue between tools.
- Automate small and big tasks.
- Provide flexibility.
- Save time and reduce errors.
- Core skill for every DevOps engineer.

SHELL SCRIPTING IS EVERYWHERE IN DEVOPS



CODE



BUILD



TEST



DEPLOY



MONITOR



ONE SIMPLE SCRIPT
CAN AUTOMATE
HOURS OF WORK.



20. BEGINNER DEVOPS AUTOMATION PROJECT

VERIQTA

BUILD ONE COMPLETE SHELL AUTOMATION PROJECT USING EVERYTHING LEARNED.

VERIQTA

PROJECT WORKFLOW

- 1 Validate Environment
- 2 Check Dependencies
- 3 Check Disk Space
- 4 Check Application
- 5 Create Backup
- 6 Deploy Application
- 7 Run Health Check
- 8 Write Logs
- 9 Return Exit Status

THIS PROJECT USES:

- ✓ Variables
- ✓ Arguments
- ✓ Conditions
- ✓ Loops
- ✓ Functions
- ✓ Logging
- ✓ Exit Codes
- ✓ Error Handling
- ✓ curl Health Check
- ✓ File Operations

KEY DEVOPS CONCEPTS COMBINED

VARIABLES

\$VAR

ARGUMENTS

\$1 \$2

CONDITIONS

if/else

LOOPS

for / while

FUNCTIONS

(){}

LOGGING

>> log

EXIT CODES

exit \$?

ERROR HANDLING

set -e

FINAL WORKING SCRIPT: deploy.sh

```
1 #!/bin/bash
2 set -euo pipefail
3 # ===== CONFIGURATION =====
4 APP_NAME=${1:-"myapp"}
5 APP_DIR=${2:-"/opt/myapp"}
6 BACKUP_DIR="/var/backups/${APP_NAME}"
7 LOG_FILE="/var/log/${APP_NAME}_deploy.log"
8 MIN_DISK_SPACE=2048 # MB
9 HEALTH_URL=${HEALTH_URL:-"http://localhost:8080/health"}
10
11 # ===== FUNCTIONS =====
12 log() { echo "[$(date +%F %T)] $1" | tee -a "$LOG_FILE"; }
13 error_exit() { log "ERROR: $1"; exit 1; }
14
15 check_command() {
16     command -v "$1" >/dev/null 2>&1 || error_exit "$1 not found"; }
17
18 check_disk() {
19     local available=$(df -Pm / | awk 'NR==2 {print $4}')
20     if [ "$available" -lt "$MIN_DISK_SPACE" ]; then
21         error_exit "Low disk space"; fi
22 }
23
24 backup_app() {
25     mkdir -p "$BACKUP_DIR"
26     tar -czf "$BACKUP_DIR/backup_$(date +%F_%T).tar.gz" "$APP_DIR"
27     log "Backup created"
28 }
29
30 deploy_app() {
31     rsync -av "$APP_DIR/" /var/www/html/
32     log "Application deployed"
33 }
34
35 health_check() {
36     local code=$(curl -s -o /dev/null -w "%{http_code}" "$HEALTH_URL" || echo "000")
37     if [ "$code" -ne 200 ]; then error_exit "Health check failed (HTTP: $code)"; fi
38     log "Health check passed"
39 }
40
41 # ===== MAIN WORKFLOW =====
42 log "Starting deployment for $APP_NAME"
43
44 log "Validating environment..."
45 check_command bash
46 check_command rsync
47 check_command curl
48 check_command tar
49 check_command df
50
51 log "Checking dependencies..."
52 check_disk
53
54 log "Checking application directory..."
55 if [ ! -d "$APP_DIR" ]; then
56     error_exit "$APP_DIR not found"; fi
57
58 log "Creating backup..."
59 backup_app
60
61 log "Deploying application..."
62 deploy_app
63
64 log "Running health check..."
65 health_check
66
67 log "Writing deployment log complete"
68 echo "SUCCESS" >> "$LOG_FILE"
69
70 # Trap unexpected errors
71 trap 'error_exit "Unexpected error on line $LINENO"' ERR
```

TROUBLESHOOTING CHECKLIST



SCRIPT WON'T RUN

- Check file permissions
`chmod +x deploy.sh`
- Check shebang
`#!/bin/bash`
- Run with: `bash -x deploy.sh`



COMMAND NOT FOUND

- Check if installed
`command -v <cmd>`
- Check PATH
`echo $PATH`
- Use full path if needed



PERMISSION DENIED

- Check file/dir permissions
`ls -l <file>`
- Use correct user
`whoami`
- Fix permissions



HEALTH CHECK FAILS

- Check URL is correct
`curl -I $HEALTH_URL`
- Check service is running
`systemctl status <svc>`
- Check firewall/network



DEPENDENCY MISSING

- Install missing tools
`apt install <tool>`
- Use check_command function
- Test on clean VM



GENERAL DEBUG TIPS

- Add set -x for tracing
`set -x`
- Read log file
`cat $LOG_FILE`
- Test each step manually

SUCCESS EXIT CODES



- exit 0 = Success
- exit 1 = General Error
- exit 2 = Validation Error
- exit 3 = Health Check Failed
- exit 4 = Disk Space Error

HOW IT WORKS TOGETHER



CHECK



BACKUP



DEPLOY



VERIFY



LOG



RESULT



KEY TAKEAWAY

This project combines all core shell scripting skills into one real-world DevOps automation workflow. Build, test, and improve it.